Práctica 3 : 5 en raya
Alicia Lozano Alcántara - B3

En esta práctica se nos ha encargado hacer 3 algoritmos(Status, Minimax, Poda alfaBeta) para poder ganarles a 4 ninjas jugando al 5 en raya.

Algoritmo Status

Este algoritmo se encarga de explorar todos los nodos existentes y devolver las opción más eficiente teniendo en cuenta, ya que estamos en un juego, las acciones también del oponente.

Mi algoritmo status está implementado de la siguiente manera, primero obtenemos todos las posibles acciones del tablero de juego pasado, si no hay más nodos hijos eso quiere decir que ese nodo es una hoja y comprobamos el estado del tablero, si el id ganador es el de mi jugador devuelve Victoria, si devuelve -1 eso quiere decir que no ha ganado nadie por lo que devolvemos Empate. En otro caso devolvemos Derrota.

Si nuestro actual tablero si tiene sucesores, exploramos cada sucesor. Cogemos uno de los hijos y volvemos a llamar a nuestra función status para que recursivamente llegue a los nodo hoja y nos devuelva un estado. Una vez en nuestro nodo actual recibimos el resultado miramos si en el tablero le tocaría jugar a mi ia o al oponente. Si es el caso de que le toca a nuestro jugador, el tablero actualiza hasta tener el mejor resultado, si es derrota el valor devuelto solo actualizamos el movimiento, si es empate y el mejor resultado no es Victoria actualizamos a empate y si el resultado actual es victoria siempre lo actualizamos y al final devolvemos el mejor resultado y mejor movimiento (ficha nueva en el tablero hijo) .
En el caso de que sea el turno del oponente sería al contrario buscaremos devolver el peor resultado posible. Así que siempre que el resultado de la función sea derrota actualizaremos.

Algoritmo MiniMax

Este algoritmo es exactamente igual que el anterior solamente que aquí en vez de tener Victoria o Derrota tendremos un heurística que nos buscará maximizar en las jugadas de mi jugador el valor y minimizar el valor en las jugadas del oponente.

La implementación al igual que el anterior se empieza comprobando si hay sucesores del tablero o es un nodo hoja o también en este caso si hemos alcanzado el máximo de profundidad. Si es el caso devolvemos un valor que calcularemos con una heurística (se explicará más adelante).

Si no es el caso como en el de status miramos de quien es la jugada en el nodo actual y actualizamos el movimiento y valor si es que el valor es el mejor que los anteriores o peor de los anteriores(caso del oponente) y cuando hayamos recorrido todos los sucesores devolvemos el movimiento y el valor.

Algoritmo Poda-AlfaBeta

El algoritmo poda alfabeta es igual que el minimax, pero en este caso si vemos que una rama no nos sirve la podamos, no la exploramos. Para ello tenemos los valor de alfa y beta que nos dirán cuándo podar.

La principal diferencia está en los condicionales, donde vamos actualizando el alfa, si es max nuestro jugador que actualizamos si es mayor valor, y el min que si es el oponente que vamos actualizando si es menor valor.

Una vez actualizado comprobamos si alfa es mayor que beta, si es el caso las demás ramas(sucesores) no habría que explorarlas , es decir , podamos.

Heurística 1

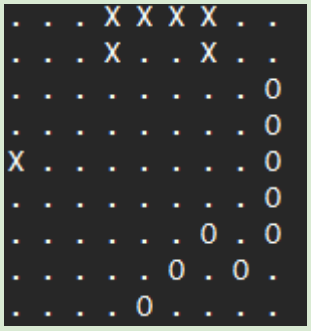
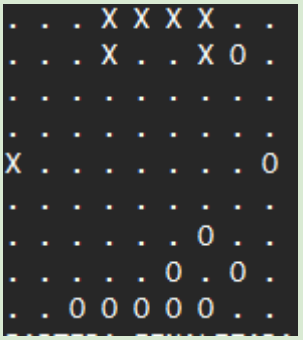
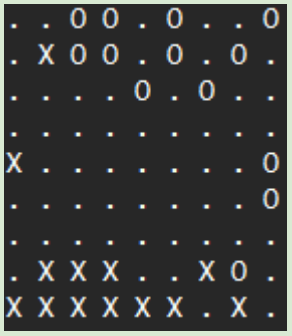
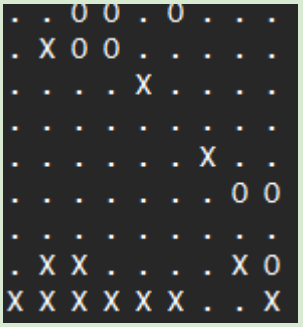
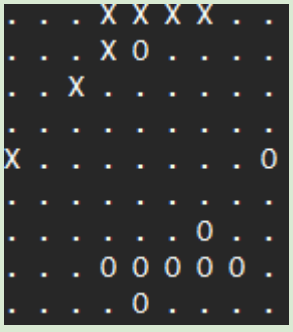
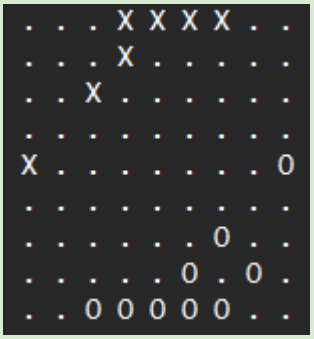
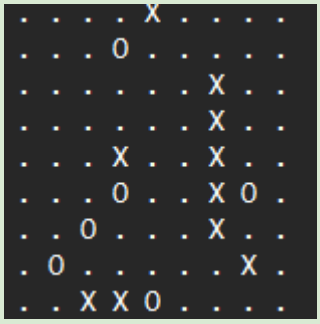
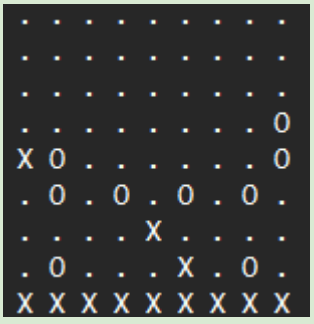
Nuestra primera heurística valora un tablero por el número de combinaciones, la longitud de fichas en secuencia de una columna , fila y ambas diagonales, es decir cuenta cuantas secuencias de x longitud hay en nuestro tablero.

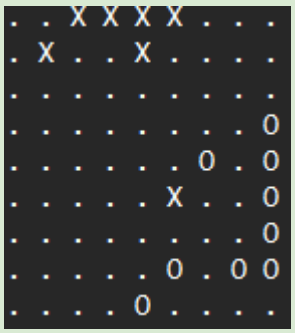
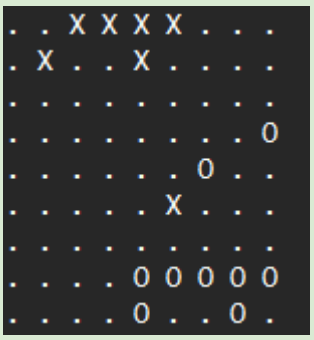
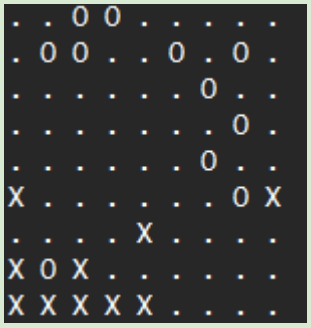
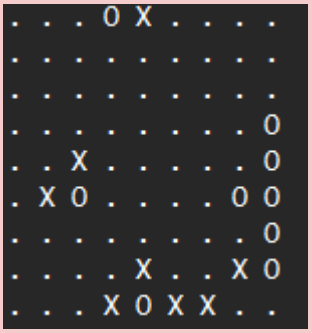
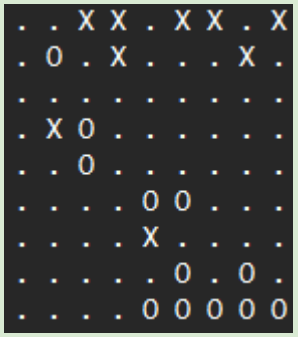
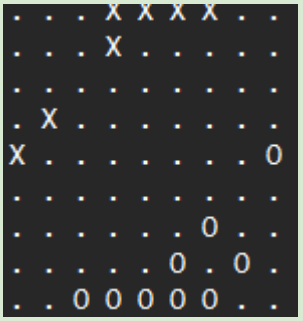
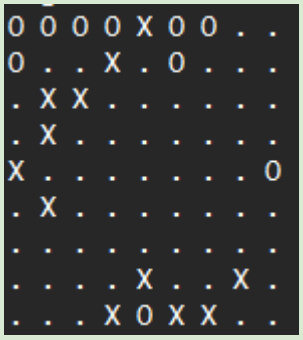
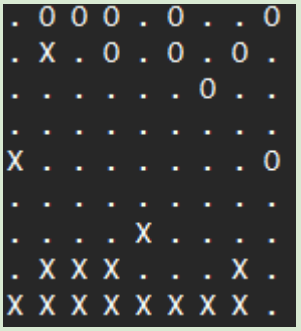
Tomamos como la puntuación del tablero la suma de los número de combinaciones desde el 4 al 2 y en cada iteración también multiplicando por 10 * valor, donde valor es la longitud al cuadrado, esto lo hacemos para que beneficie si tenemos una secuencia más larga.
Y además puntuamos el tablero negativamente con que el oponente tenga combinaciones, que lo calculamos igual pero dándole un poco más de valor para así hacer que nuestro jugador priorice obstaculizar al oponente si es que va ganando

Heurística 2

La segunda heurística no difiere mucho de la primer solo que en este caso no contamos puntos negativos, es decir nunca nos llegamos a plantear las fichas que tiene el rival por lo que nunca se va a plantear bloquear al oponente, que eso es lo que nos hace perder en algunos casos, ya que siempre va a buscar hacer una secuencia más grande, llegar a 5 pero puede ser que antes que lo consigamos el oponente ya haya ganado la victoria

Comparación de heurísticas

	HEURÍSTICA 1	HEURÍSTICA 2
NINJA1 VS INTELIGENTE	 Gana el jugador 2	 Gana el jugador 2
INTELIGENTE VS NINJA1	 Gana el Jugador 1	 Gana el Jugador 1
NINJA2 VS INTELIGENTE	 Gana el Jugador 2	 Gana el Jugador 2
INTELIGENTE VS NINJA2	 Gana el Jugador 1	 Gana el Jugador 1

NINJA3 VS INTELIGENTE	 Gana el Jugador 2	 Gana el Jugador 2
INTELIGENTE VS NINJA3	 Gana el Jugador 1	 Gana el Jugador 2
NINJA4 VS INTELIGENTE	 Gana el Jugador 2	 Gana el Jugador 2
INTELIGENTE VS NINJA4	 Gana el Jugador 1	 Gana el Jugador 1

Como podemos observar en las diferencias solo hay uno que no lo pase, y el que no ha pasado es porque nuestro jugador no obstaculiza al otro jugador.